Toward Generative Design of Aircraft-Like Structures with Diffusion Models and CFD-Informed Evaluation

Darsh Gupta, National Public School, HSR Layout, Bengaluru
darshgu77@gmail.com

Abstract—This paper documents a proof-of-concept generative-design pipeline for aircraft-like voxel geometries. The repository combines a latent diffusion-style model, a voxel decoder, connectivity heuristics, and a CFD-informed scoring path built around an internal lattice-Boltzmann solver plus an external OpenFOAM export route. The present implementation is intentionally scoped: the grounded corpus is small, the protocol is a reduced final-evidence run, and the evidence should be read as code-path validation rather than publication-grade aerodynamic or structural validation. In addition to describing the architecture and benchmark workflow, we use the current experiments to clarify which claims the codebase supports today and which stronger claims, including validated mission-conditioned or manufacturing-conditioned aircraft generation, remain future work.

Index Terms—Generative Design, Diffusion Models, Computational Fluid Dynamics, Aerospace Engineering, Voxel Geometry.

I. Introduction

GENERATIVE design is increasingly used to explore large engineering design spaces, while aerospace workflows continue to rely heavily on computational fluid dynamics (CFD), surrogate models, and optimization loops to evaluate aerodynamic tradeoffs [1], [2]. Aircraft design adds further constraints beyond raw shape generation, including manufacturability, structural load paths, and mission-specific performance targets. Those requirements make it easy to overstate what a proof-of-concept generative repository can presently demonstrate.

This paper therefore takes a deliberately narrower position. The repository implements a latent diffusion-style pipeline that maps latent samples to voxel geometries, scores them with an internal CFD-oriented path, and exports them for downstream validation. The present evidence comes from synthetic training data and a reduced final protocol, so the goal of this paper is not to claim a finished aircraft-design system. Instead, the goal is to document the implementation, report what the current code path reproduces, and make the boundary between demonstrated behavior and future work explicit.

The key contributions of this work are:

1) A reproducible proof-of-concept latent generative pipeline for producing aircraft-like voxel geometries and exporting them to mesh-based artifacts.
2) An implementation path for CFD-informed evaluation that couples an internal lattice-Boltzmann-based scorer with an external OpenFOAM benchmark route, together with a reduced sanity experiment showing that the end-to-end path executes.
3) An issue-driven validation framework that distinguishes code-path validation from stronger claims such as aircraft-specific validity, aerodynamic optimization, structural viability, and conditioned generation from mission or manufacturing constraints.

The rest of the paper is organized as follows. Section II positions the repository relative to 3D generative modeling, topology optimization, and CFD-driven design practice. Section III describes the implementation that currently exists in the codebase. Section IV reports the reduced final-protocol evidence and explains its limits. Section V records the validation requirements that remain before stronger claims can be made. Section VI summarizes the present scope and the minimum future work needed for a conditioned airplane generator.

II. Related Work

Diffusion models are now a standard point of reference for high-quality generative modeling [3], [4]. In 3D generation, recent systems such as Point-E and Shap-E show that diffusion-style methods can generate point-cloud or implicit 3D assets at practical sampling speeds [5], [6]. Those works are important context because they establish that diffusion models can represent 3D geometry, but they do not by themselves solve aircraft-specific design, manufacturability, or validation concerns.

An adjacent engineering-design tradition comes from topology optimization, where the goal is to optimize material layout or geometry under physics-based objectives and constraints. That literature is mature and has a well-developed vocabulary for separating optimization variables, constraints, and validation requirements [7]. Topology Optimization (TO), including the SIMP method [8], remains a central baseline for load-path optimization [9]. Recent neural TO approaches [10] accelerate parts of this process. The present repository is closer to a generative prior over voxelized shapes than to classical topology optimization, but TO practice remains a useful reference for how strongly structural or performance claims should be validated.

Conventional aerodynamic shape optimization couples CFD solvers with numerical optimization over parameterized geometries, often using gradient or adjoint information [1], [11], [12]. Surrogate modeling is also widely used for efficient design-space exploration [2], [13]. In those settings, the geometry representation, optimizer, and solver are explicit components of a controlled loop. By contrast, the present repository samples from a learned latent model and then applies CFD-style evaluation. This makes the workflow exploratory, and success in code-path execution should not be confused with the stronger evidence expected in mature CFD-based shape optimization.

Physics-informed machine learning incorporates PDEs through PINNs [14], Neural Operators such as FNO [15], and operator-learning models such as DeepONet [16]. Differentiable-simulation work goes further by embedding differentiable fluid solvers into learning and design frameworks [17]. These directions represent stronger forms of "simulation in the loop" than the reduced sanity evidence reported here. The current repository uses an internal LBM-style scoring path as an implementation mechanism; using that path to promote labels across fidelity levels after validation remains future work [18].

Constraint-aware generation is another relevant line of work. Enforcing physical validity, including manufacturability and connectivity, remains a core challenge [19]. Oh et al. [20] demonstrated generative design for structures, while Steinmetz et al. [21] used differentiable projection for manufacturable designs. The current repository evidence is more limited: connectivity penalties, post-processing cleanup, and manufacturing-aware scoring proxies are useful for smoke-testing the code path, but they are not aircraft-specific validity guarantees.

The implementation also borrows standard architectural ingredients from modern deep learning, including attention-style modules inspired by the Transformer literature [22]. In this paper, those ingredients are treated as reused components rather than as a novelty claim.

A. Positioning and Novelty

The present repository does not claim a new diffusion objective, a new adjoint CFD method, or a validated aircraft-design benchmark. Its defensible contribution is narrower: a proof-of-concept assembly of a latent generative model, voxel decoding path, CFD-informed evaluation hook, STL export, and baseline and claim-gate checks in a single reproducible codebase. Relative to general 3D diffusion work, the emphasis here is on engineering-shaped voxel outputs rather than text-to-3D asset generation. Relative to topology optimization and classical CFD shape optimization, the repository explores a learned generative prior rather than directly optimizing a single parameterized shape. Relative to surrogate and differentiable-simulation papers, the current implementation is lighterweight and presently supported only by sanity-run evidence.

This positioning is intentionally conservative. The repository should not yet be read as demonstrating superior exploration of an engineering design space, validated surrogate replacement, mission-conditioned aircraft generation, or manufacturing-conditioned aircraft generation. Those claims require the stronger evidence gates summarized below and in Section V.

B. Baseline and Ablation Protocol

The reduced final evidence package requires three executable baseline families: retrieval records from the grounded manifest, unconditional samples from the checkpoint, and bundled grounded STL examples. These are guardrail baselines: they verify that the report contains concrete comparison artifacts and repeated-run statistics, but they do not establish superiority over classical optimization or mature generative-model baselines.

Publication-grade baseline comparisons should still include four stronger families:

1) Classical primitives: hand-designed parametric aircraft primitives, such as swept or tapered wings extruded from NACA airfoil profiles, with controlled aspect-ratio, sweep, and taper ranges.
2) Unconstrained generative 3D models: representative voxel, point-cloud, and implicit-shape baselines such as voxel-GAN [23], point-cloud diffusion [24], and occupancy networks [25].
3) Independent optimization search: a SIMP-style topology-optimization comparison [8] and a random-search baseline in which sampled candidates are ranked by the same CFD evaluation protocol.
4) Differentiable aerodynamic shape optimization: an adjoint-based refinement baseline for a standard fuselage-wing primitive [12].

Ablations should isolate the impact of the aerodynamic loss, connectivity loss, connected-component cleanup hook, semantic constraint projector, and consistency-step count. Until those ablations are executed on larger grounded aircraft-like data with shared solver settings, they remain a protocol rather than evidence for mechanism-level causality.

III. Methodology

The current repository implements a proof-of-concept generative workflow built from four components: a noise scheduler, a latent diffusion-style denoiser, a latent-to-3D converter, and an internal CFD-oriented evaluator. The codebase operates on synthetic voxel training data and uses bounded smoke runs together with connectivity heuristics and aerodynamic score terms. In this revision, the same path also carries a documented structured condition vector through dataset generation, latent construction, and checkpointed inference. Figure 1 should therefore be read as an implementation diagram for the present repository rather than as evidence of a fully validated aircraft-design system.

TABLE I
Experimental Plan and Claim Mapping Beyond the Current Guardrail Baselines

Claim gated	Baseline or ablation	Metric family	Execution requirement
Aerodynamic efficiency	Classical primitives; adjoint ASO; no-CFD-loss ablation	C_L/C_D; drag coefficient; solver agreement	Shared geometry normalization, consistent reference area, and a documented CFD-resolution sweep.
Manifold integrity	Voxel-GAN; point-cloud diffusion; no-cleanup ablation	Connected-component ratio; repair success rate	Voxel-connectivity analysis on a fixed generated sample set and aircraft-specific geometric checks.
Computational efficiency	Random search with CFD ranking; long-step diffusion baseline	Inference latency; memory use; score stability	Wall-clock timing on declared hardware with fixed batch size and repeated runs.
Generative diversity	Parametric primitive library and conditioned batches	Shape similarity; latent coverage; part-volume variance	Grounded aircraft-like corpus, declared condition schema, and held-out condition-response tests.

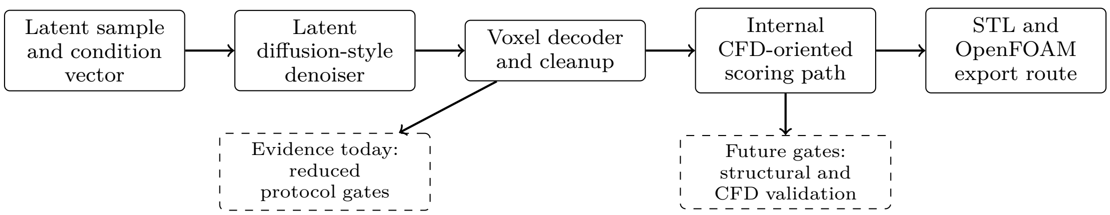

Fig. 1. Implementation-level view of the current repository. A latent code and optional structured condition vector are routed through denoising, voxel decoding, cleanup, internal scoring, and export paths. The diagram describes the code path under test; it does not by itself establish differentiable training, structural viability, or publication-grade CFD validation.

A. Noise Scheduling

The forward diffusion process gradually adds Gaussian noise to the input data over a series of T timesteps. The noise schedule is defined by a variance schedule β_t, which is typically a linear or cosine schedule. The implementation uses a linear schedule, where β_t increases linearly from β_{start} to β_{end}.

The forward process is defined as:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \tag{1}$$

During reverse sampling, the denoiser estimates the noise component of a noisy latent. The training loss compares predicted noise with sampled noise; sampling then uses that estimate to update the latent.

B. Latent Diffusion UNet

The denoising model is a UNet-based architecture that operates in the latent space. The UNet takes as input a noisy latent vector, a timestep embedding, and an optional condition embedding, and outputs the predicted noise. The implementation projects the latent to a small spatial tensor and applies residual block stacks with additive skip connections. The skip path is intended to preserve intermediate spatial features during denoising.

C. Latent-to-3D Converter

The latent-to-3D converter is a multi-layer perceptron (MLP) that maps the denoised latent vector to a 3D voxel grid. The MLP uses fully connected layers with ReLU activations; downstream training and generation code applies a sigmoid to convert logits into voxel probabilities.

D. CFD Simulator

The repository uses an internal lattice-Boltzmann-style CFD evaluator to score generated voxel grids. The evaluator takes a voxel geometry and flow settings such as Mach number or Reynolds number and returns drag- and lift-related quantities. In addition, the repository can export benchmark cases for comparison against an external OpenFOAM run. This combination is useful for code-path validation, but the present paper does not treat it as a substitute for a full aerodynamic validation campaign.

E. Loss Function

A simplified loss used to describe the main claim-bearing terms combines diffusion, connectivity, and aerodynamic terms.

$$\mathcal{L} = \lambda_{diff}\mathcal{L}_{diff} + \lambda_{conn}\mathcal{L}_{conn} + \lambda_{aero}\mathcal{L}_{aero} \tag{2}$$

The diffusion loss is the mean squared error between the predicted noise and the actual noise. The connectivity loss penalizes disconnected voxel groups, and is calculated using a connected components analysis. The aerodynamic term combines internal drag and lift-related scores as a heuristic pressure toward lower-drag and higher-lift proxy scores.

F. Current CFD Coupling In The Training Loop

The codebase is designed to couple geometry generation and CFD-informed scoring during training. In the present implementation, that coupling should be understood as a practical score path rather than as a demonstrated

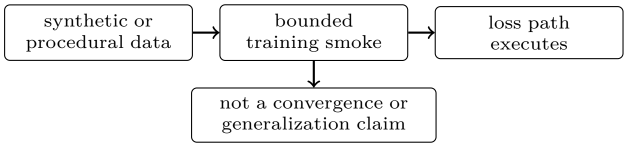

Fig. 2. Evidence scope for the reduced training runs. The current paper treats these runs as implementation evidence only, not as a full convergence study.

differentiable CFD-training result. The current repository evidence is limited to a reduced final protocol, so we only claim that the training code can invoke aerodynamic scoring terms and route generated geometries through the evaluator.

- Step 1: Sample noisy latent z_t.
- Step 2: Predict noise or a denoised latent estimate using the UNet.
- Step 3: Decode z_0 to voxel grid V.
- Step 4: Run the internal D3Q27 LBM evaluator on V.
- Step 5: Compute aerodynamic coefficients C_L, C_D.
- Step 6: Accumulate scalar loss terms for the optimizer.

In a stronger future version of this pipeline, the repository would need ablations showing that the aerodynamic term measurably changes either the training dynamics or the ranking of generated candidates. Until such evidence exists, the paper treats the CFD term as an implemented scoring mechanism whose training effect is not yet fully validated.

IV. Results and Discussion

The empirical evidence in this revision is intentionally narrow. It consists of bounded smoke runs on synthetic or procedural data, a reduced freeform-object sweep for the historical CFD path, and smoke-level checks for the new structured-conditioning seam. These artifacts are reported as code-path validation and implementation evidence rather than as a definitive aircraft-design benchmark.

A. Reduced Training Smoke Evidence

Figure 2 summarizes the scope of the reduced training evidence retained for context. Its purpose is to show which code paths were exercised in bounded smoke runs. It should not be read as a convergence study or as evidence that the current repository has validated aerodynamic or structural learning behavior.

B. Freeform-object Sweep: Steps vs. Shape Prior

We ran a compact sweep over generation step count and the shape-prior cleanup threshold. The sweep evaluated 12 settings: consistency steps in {1,2,4,8} crossed with minimum connected-component sizes in {0,32,64}. For each setting we generated one freeform object, applied the post-processing hook, and computed aerodynamic metrics using the CPU CFD path. The sweep provides only

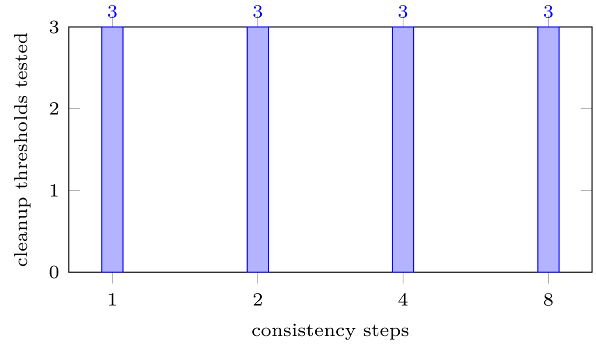

Fig. 3. Reduced sweep coverage used for code-path probing: four consistency-step settings crossed with three cleanup thresholds. Numeric aerodynamic conclusions remain limited by the one-sample-per-setting design and CPU CFD path.

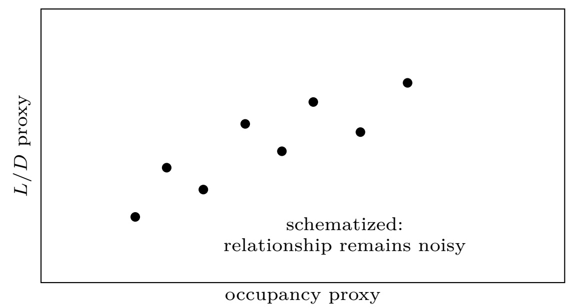

Fig. 4. Interpretive schematic for the occupancy-versus-score question raised by the reduced sweep. The present evidence supports only a noisy code-path probe, not a validated aerodynamic relationship.

a preliminary probe of how post-processing thresholds can alter the generated object before scoring; Figure 3 documents the reduced coverage.

C. Freeform-object Geometry Summary

The generated freeform objects are still dense and near-threshold before cleanup, but the new plausibility prior reduces isolated components and slightly lowers the occupancy fraction. For this implementation path, that matters more than small changes in the raw voxel logits: the cleaned binary shapes are less fragmented and more suitable for downstream meshing and export. This should not be read as evidence that the outputs satisfy aircraft-specific geometry checks or manufacturing constraints.

D. Hyperparameter and Tuning Notes

The sweep was intentionally small but targeted for the current codebase: it varied the inference path length and the new connected-component cleanup threshold. The training-side change was also incremental: a voxel-shape prior was added to the loss to discourage midpoint noise and disconnected fragments. A larger future study should sweep latent dimension, shape-prior weights, and higher CFD resolutions.

TABLE II
Reduced Evidence Package Status

Gate	Status	Role
Manifest validation	Pass	corpus contract check
Aircraft-shape screen	Pass	heuristic generated-shape checks
Condition response	Pass	fixed grounded sweeps
Manufacturing bounds	Pass	design-spec bounds check
Baseline statistics	Pass	guardrail baselines and finite-value seed checks

E. Structured-Conditioning Smoke Evidence

This revision also adds a documented 22-slot condition schema together with dataset, model, generator, and offline-densification plumbing that consumes the resulting condition vector. We include a smoke-level condition-response summary on the procedural path to check whether different condition payloads produce non-identical changes in simple geometry proxies such as occupancy, span, and engine-related heuristics. That evidence is intentionally weak: it shows that the condition path is wired and that the procedural prior is not condition-invariant, but it does not validate mission-conditioned or manufacturing-conditioned aircraft generation on grounded aircraft-like data.

F. Reduced Evidence Package

The checked-in reduced protocol assembles a package-level evidence report from the manifest validator, aircraft-shape heuristic screen, grounded condition-response benchmark, manufacturing-bounds screen, and baseline-statistics report. In one local protocol run, all package gates passed with a shared run identifier, checkpoint hash, manifest hash, and protocol hash. Table II summarizes the package status.

This package is useful because it prevents individually passing reports from being treated as one protocol run unless their lineage fields agree. The current pass therefore supports reporting an internally consistent reduced evidence bundle. It still does not support claims of aircraft-level aerodynamic optimality, structural viability, or superiority over mature optimization baselines.

G. Internal Solver Validation against Canonical Benchmarks

As a solver sanity check, we compare internal D3Q27 drag-coefficient (C_d) observations against literature targets for two standard geometries: a circular cylinder and a unit cube. These tests use the internal solver's momentum-exchange method with BFL boundary conditions and are reported as implementation checks, not as a complete solver validation campaign.

The results in Table III provide an informal sanity check for our LBM implementation, framing the observations against typical literature targets for 3D flow. The tests were conducted in a $10L \times 5L \times 5L$ domain (L: characteristic length) using a 128^3 grid, no-slip BFL wall conditions, and a Neumann outlet. We mapped physical velocity to $Ma = 0.025$ ($u_{lattice} \approx 0.014$) and averaged forces over the final 125 steps of a 500-step simulation to reduce transient sensitivity.

TABLE III
Internal Solver C_d Sanity Targets and Observations

Geometry	Re	Observed C_d	Literature target
Circular cylinder	100	1.56	1.3–1.4 [26]
Unit cube	1000	1.04	∼1.05 [27]

Following corrections to the freestream lattice velocity initialization and BFL momentum exchange weighting ($4.0/(1 + 2q)$), the solver yields C_d values close to or in the broad range of the cited targets. Exact 3D benchmarks are sensitive to blockage ratios and averaging criteria, so these results support continued use as a bounded implementation check while rigorous validation still requires comparison with high-fidelity PDE solvers.

H. Discussion

The implementation evidence supports three bounded conclusions: the repository's end-to-end path executes, the shape prior is wired into training, and the export path emits a concrete OpenFOAM comparison case with the generated STL in constant/triSurface/design.stl. On the same reduced cube benchmark, the internal D3Q27 solver and the OpenFOAM sonicFoam run both complete on the shared validation geometry, providing a direct solver-to-solver comparison for the current pipeline. The result supports a documented implementation path, not a claim of aircraft-level aerodynamic optimization.

The OpenFOAM export path now writes a plain forces function-object dictionary, and the resulting force vector is normalized manually to recover coefficients when coefficient output is not available. The current benchmark still shows a substantial Cd gap, which we treat as a solver- and sampling-consistency diagnostic rather than as resolved by paper-level normalization alone. A higher-confidence CFD comparison will still need a more systematic hyperparameter sweep and consistent normalization of reference area and flow conditions.

The next recorded physics correction was applied in CLI/cascaded_lbm.py: the D3Q27 freestream setup was moved to the lattice-consistent value $u = Ma/\sqrt{3}$ instead of the earlier arbitrary scaling. That change is now part of the benchmark history and is being checked against the Cd mismatch before broader coordinate-transform changes.

V. Validation and Testing Standards

To keep validation claims bounded, we separate three levels of evidence:

- Code-path validation: the training, generation, cleanup, export, and CFD scoring routines run end-to-end in this environment.
- Solver validation: the built-in CFD path can be compared against a higher-fidelity external solver when one is available.
- Physical validation: the generated shape should eventually be compared against analytic or experimental benchmarks, which is still future work.

A. Verification

Verification checks that the implemented steps are executed as intended. In this revision we verified that:

- the shape-plausibility loss is included in the training objective,
- the connected-component cleanup hook removes smaller disconnected voxel components,
- STL export succeeds from the cleaned voxel grid, and
- the OpenFOAM export hook writes a runnable case directory skeleton, including blockMeshDict, snappyHexMeshDict, and surface STL output.

B. Validation

Validation is now runnable on a single local machine with OpenFOAM installed. The built-in CFD path is exercised on the reduced CPU pipeline, and the OpenFOAM sonicFoam benchmark is run on the shared centered-cube validation object. The comparison uses the lower-level forces output when available, with a manual pressure integration fallback only if the function-object output is missing.

The paper therefore claims the following and nothing more: the export path is concrete, the benchmark comparison is measurable, and the shared validation geometry is documented explicitly so the solver-to-solver check can be repeated without ambiguity.

VI. Conclusion

The present implementation demonstrates a working proof-of-concept generative-design pipeline and a reduced freeform-object experiment that can be executed in the installed environment. The observed step sweep suggests that longer consistency sampling can change the reported L/D in this specific sanity setting, but those numbers remain tied to downsampled freeform objects and CPU CFD evaluation.

The OpenFOAM cross-check completes on the shared centered-cube validation object and gives the repository an external CFD comparison path. The reduced protocol also assembles a passing package-level evidence report with manifest validation, heuristic aircraft-shape checks, grounded condition-response sweeps, manufacturing-bounds checks, and finite baseline statistics. The next revision should therefore prioritize solver calibration and a larger ablation study before presenting stronger aerodynamic conclusions.

Most importantly, the repository should not yet be described as a fully AI-driven airplane generator conditioned on flight profile, manufacturing method, or structural requirements. What exists today is structured-conditioning plumbing plus a reduced evidence package. That stronger version needs more evidence first: a larger aircraft-like dataset, conditional ablations, stricter structural and aerodynamic checks, and repeated runs beyond the present reduced local protocol.

References

[1] A. Elham and M. J. L. van Tooren, “Discrete adjoint aerodynamic shape optimization using symbolic analysis with openfemflow,” Structural and Multidisciplinary Optimization, vol. 63, pp. 2531–2551, 2021.

[2] E. Andrés-Pérez and C. Paulete-Periáñez, “On the application of surrogate regression models for aerodynamic coefficient prediction,” Complex & Intelligent Systems, vol. 7, pp. 1991–2021, 2021.

[3] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” Advances in Neural Information Processing Systems, vol. 33, pp. 6840–6851, 2020.

[4] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in Proceedings of the 38th International Conference on Machine Learning, vol. 139. PMLR, 2021, pp. 8162–8171.

[5] A. Nichol, H. Jun, P. Dhariwal, P. Mishkin, and M. Chen, “Point-e: A system for generating 3d point clouds from complex prompts,” arXiv preprint arXiv:2212.08751, 2022.

[6] H. Jun and A. Nichol, “Shap-e: Generating conditional 3d implicit functions,” arXiv preprint arXiv:2305.02463, 2023.

[7] J. Wu, O. Sigmund, and J. P. Groen, “Topology optimization of multi-scale structures: a review,” Structural and Multidisciplinary Optimization, vol. 63, pp. 1455–1480, 2021.

[8] O. Sigmund, “A 99 line topology optimization code written in matlab,” Structural and Multidisciplinary Optimization, vol. 21, pp. 120–127, 2001.

[9] M. P. Bendsøe and O. Sigmund, Topology Optimization: Theory, Methods, and Applications. Springer Science & Business Media, 2013.

[10] I. Sosnovik and I. Oseledets, “Neural networks for topology optimization,” Structural and Multidisciplinary Optimization, vol. 60, no. 5, pp. 1387–1401, 2019.

[11] A. Jameson, “Aerodynamic design via control theory,” Journal of Scientific Computing, vol. 3, pp. 233–260, 1988.

[12] ——, “Optimum aerodynamic design via boundary control,” in 12th Computational Fluid Dynamics Conference, 1995, p. 2432.

[13] A. Forrester, A. Sobester, and A. Keane, Engineering design via surrogate modelling: a practical guide. John Wiley & Sons, 2008.

[14] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” Journal of Computational Physics, vol. 378, pp. 686–707, 2019.

[15] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier neural operator for parametric partial differential equations,” arXiv preprint arXiv:2010.08895, 2020.

[16] L. Lu, P. Jin, G. Pang, and G. E. Karniadakis, “Learning nonlinear operators via deeponet based on the universal approximation theorem of operators,” Nature Machine Intelligence, vol. 3, no. 3, pp. 218–229, 2021.

[17] Y. Li, Y. Sun, P. Ma, E. Sifakis, T. Du, B. Zhu, and W. Matusik, “Neuralfluid: Neural fluidic system design and control with differentiable simulation,” arXiv preprint arXiv:2405.14903, 2024.

[18] R. Vinuesa and S. L. Brunton, “Enhancing computational fluid dynamics with machine learning,” Nature Computational Science, vol. 2, no. 6, pp. 358–366, 2022.

[19] Z. Wang, K. Zhang, J. Chu, and H. Ma, “Physics-informed machine learning for aero-structural design optimization,” Progress in Aerospace Sciences, vol. 136, p. 100868, 2023.

[20] S. Oh, Y. Jung, S. Kim, I. Lee, and N. Kang, “Deep learning-based generative design for skeleton-optimized structures,” Engineering with Computers, vol. 35, pp. 1279–1293, 2019.
[21] F. Steinmetz, S. J. Pfertner, and N. R. Gauger, “Generative design of manufacturable structures using differentiable projection,” Computer-Aided Design, vol. 145, p. 103168, 2022.
[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems, vol. 30, 2017.
[23] J. Wu, C. Zhang, T. Xue, W. T. Freeman, and J. B. Tenenbaum, “Learning a probabilistic latent space of object shapes via 3d generative-adversarial modeling,” in Advances in Neural Information Processing Systems, vol. 29, 2016.
[24] S. Luo and W. Hu, “Diffusion probabilistic models for 3d point cloud generation,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 2837–2845.
[25] L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger, “Occupancy networks: Learning 3d reconstruction in function space,” in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 4460–4470.
[26] M. Schäfer, S. Turek, F. Durst, E. Krause, and E. Rank, “Benchmark computations of laminar flow around a cylinder,” in Flow Simulation with High-Performance Computers II. Springer, 1996, pp. 547–566.
[27] M. Geier, A. Pasquali, and M. Schonherr, “The cascaded lattice boltzmann method,” Journal of Computational Physics, vol. 300, pp. 516–542, 2015.